ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/316736515

Deep Neural Networks for Text: A Review

Conference Paper · May 2017

CITATIONS
3

READS
1,740

4 authors, including:

Chiung Ching Ho
Sunway University
108 PUBLICATIONS **426** CITATIONS

SEE PROFILE

Khairul Nizam Baharim
Petroliam Nasional Berhad
7 PUBLICATIONS **19** CITATIONS

SEE PROFILE

Ahmad Abdulsalam
University of Ilorin
1 PUBLICATION **3** CITATIONS

SEE PROFILE

All content following this page was uploaded by Chiung Ching Ho on 08 May 2017.

The user has requested enhancement of the downloaded file.

Deep Neural Networks for Text: A Review

Chiung Ching Ho[1], Khairul Nizam Baharim[2], Ahmad Abdulsalam Ahmad Fatan[2] , and Mohd Shafiq Bin Alias[2]

[1]Multimedia University, Data Science Institute,
[2]TM Research & Development
Corresponding author's email : ccho@mmu.edu.my
{khairulnizam,abdulsalam,shafiq}@tmrnd.com.my

***Abstract*:** Deep learning is a powerful technique for learning representation and can be used to learn features within text. The learned features are useful for solving Natural Language Processing problem. In this paper we review key literature related to deep learning and its application on solving text analysis.

***Keywords*:** Deep Learning, Text Analysis, CNN, RNN

1. Introduction

Text data has been an important source of information, and continues to provide information in ever increasing volumes. Deep learning has been applied successfully for the purpose of learning features of text, and can be applied to enable many Natural Language Processing problem. In this paper, we review the application of deep learning onto text, focusing on the convolutional neural network (CNN) architecture. This paper is divided into the following sections: 1.0 Introduction, 2.0 What is Deep Learning , 3.0 Deep Learning and Text, 4.0 Convolution Neural Network and Text, 5.0 Recurrent Neural Network and Text, and 6.0 Conclusion.

2. What is Deep Learning

Deep learning [1] first came to public attention when it was used to win the 2012 ImageNet Large-Scale Visual Recognition Challenge with a top-5 test error rate of 15.4% [2]. Recently, AlphaGo used DL to defeat a human professional Go player which was an unprecedented achievement [3].

Deep learning is being increasingly considered as a subset of machine learning, both supervised and unsupervised. In machine learning, human-engineered representations and input features are important to ensure that the machine learning task is performed with high accuracy. The various machine learning algorithms optimizes the weights for the representation and input features to make the best final prediction. In practice, machine learning can thus be divided into two aspect. The first aspect calls for domain-specific subject-matter expertise to describe the features in a way that is suitable for computing. The second aspect is to optimize the weights of these features via a learning algorithm.

The deep learning approach builds on the aforementioned ideas by attempting to automatically learn good features or representations, in a process described as representation learning. In the course of representation learning, multiple-levels of representation is learned from 'raw' inputs (for example, text) to construct an output. The accuracy of the learned representation is driven by automatically adjusted weights.

The application of deep learning as a tool for solving machine learning problems have several advantages over techniques which are highly dependent on human engineered features. Human engineered features are very often highly specific, and are incomplete due to latent patterns which are not detected even by subject matter experts. The process of engineering features itself takes a tremendous amount of time to design and validate. Learned representations and features takes relatively lesser amount of time to be learned, and are easily adaptable. In terms of generalization, the deep learnable framework is flexible and is useful for learning real-world, time-dependent, visual and linguistic information. Deep learning facilitates both unsupervised learning (from raw text for example) and supervised learning (text which are labelled). Applications of deep learning in problem solving has been increasing of late due to the following factors: the growth of available data, advances in multicore CPUs and GPUs which facilitates deep learning, and the increase of new models and algorithms. Figure 1 depicts a generic deep learning network which has n-layers. The layers comprise one input layer, m-hidden layers and one output layer.

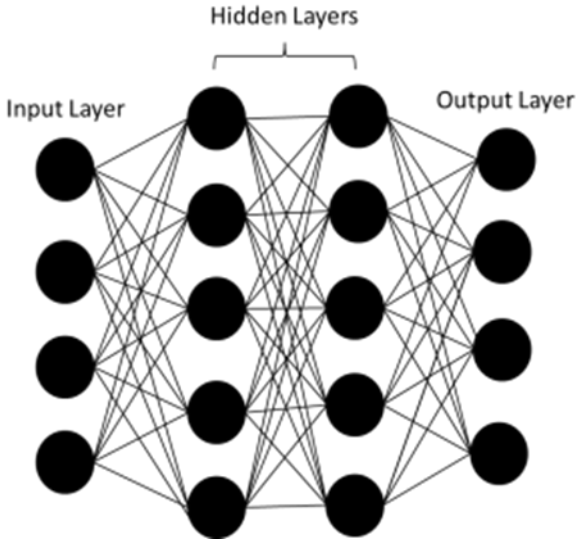

Figure 1: Generic deep learning network [4]

In this paper, we focus our discussion for deep learning on deep neural networks, which is the dominant model within deep learning. We also choose to focus the discussion on natural language processing, instead of a historical accounting of deep neural networks [5].

3. Deep Learning and Text

The initial successes of using deep learning to solve image recognition problems has led to efforts to use deep learning for learning features in other domains. In this section, we will explore how deep learning has been applied for solving problems in text. Text mining or processing is a subset of natural language processing (NLP), while deep learning can effectively solve syntactic analysis and semantic interpretation of text. The application of deep learning for solving text related problems tries to solve of the fundamental problem of NLP, which is the inherent complexity in representing, learning and using linguistic knowledge which is often influenced by contextual and situational real-world settings.

Deep learning has been used to representation the morphology of text, or in other words the representation of the constituent parts of a word. Without considering morphology, rare and complex words are represented poorly as a vector representation. [6] used a Recurrent Neural Network (RNN) to treat word morpheme as a vector, which resulted in improved results over non deep learning techniques in terms of estimation of rare and complex words. Figure 2 shows how a morpheme is represented as a vector and how the RNN is used to combine two morpheme vectors into a singular vector.

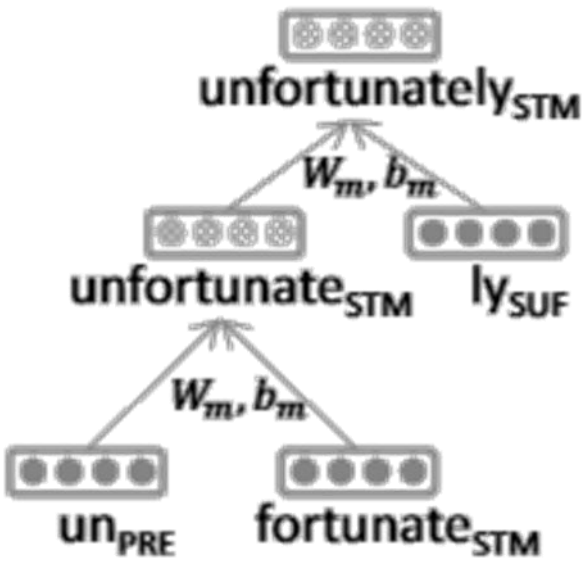

Figure 2: Morpheme representation as vectors [6]

This idea of representing text via deep learning also addresses the problems of representing the syntax of text. In traditional NLP, an individual phrase is divided into distinct categories such as noun and verb phrases. Utilizing deep learning neural networks, each word and sentence is expressed as a vector. Figure 3 shows how a sentence is represented as a vector.

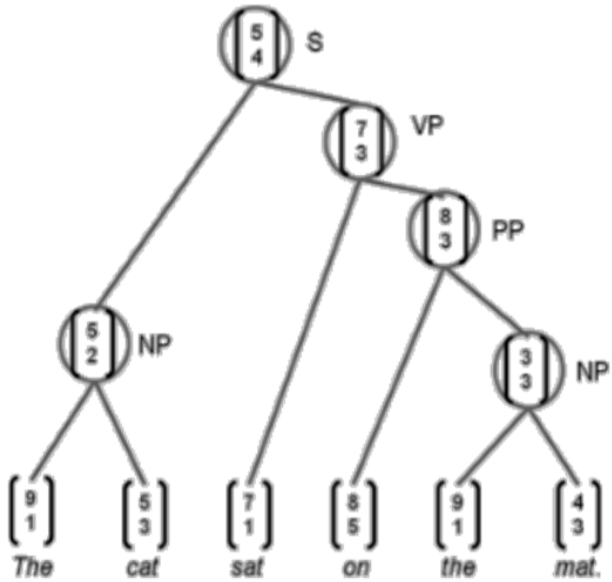

Figure 3: Representation of the syntax of a sentence as a vector [7]

In the area of text semantics, the traditional approach utilizes lambda calculus to represent the semantics of text. The representation of semantics takes the form of carefully engineered functions, which takes as inputs other functions. In this approach, the fuzziness and similarity inherent in the semantics of language is ignored. The deep learning approach treats the semantic representation of text in a different way. Every word, phrase and logical expression in a body of text is represented as a vector. These vectors are aggregated into a single vector. [8] investigated the usage of RNN in learning logical relationship (for example, entailment and contradiction) between two sentences.

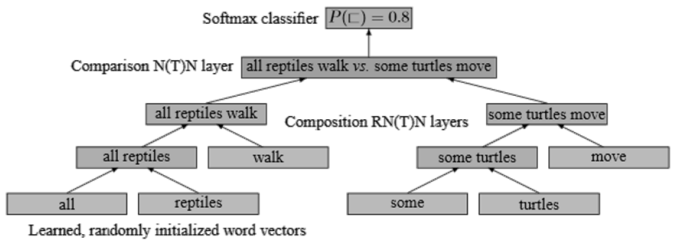

Figure 4: RNN network is used to compare the semantic value of two different sentences [8]

4. Convolutional neural network and text

The 1-d convolution processes inputs viewed as a sequence s ∈ R, s, as well as a vector of weights m ∈ R, m. The convolution's filter is the vector m. s represents the sentence submitted as an input and si ∈ R is a single feature value representative of the sentence's i-th word. The 1-d convolution is the result of the scalar product of the vector m with each m-gram in the sentence s to obtain another sequence c:

$$cj = m^T s_{j-m+1:j} \quad (1)$$

Equation 1 can result in two different forms of convolution which is dependent on the index j, either narrow or wide. The narrow type of convolution is true provided that $s \geq m$ and results in sequence $c \in R^{s-m+1}$ with j ranging from m to s. The wide type of convolution is not dependent on s or m and results in a sequence $c \in R^{s+m-1}$ where the index j is range bound from 1 to $s + m - 1$. Out-of-range input values for si is where $i < 1$ or $i > s$ are taken to be zero. The result of the narrow convolution can treated as a sub-sequence of the result of the wide convolution.

Convolutional neural network (CNN) consist of several layers of convolution with non-linear activation functions such as TanH or ReLu which are then used unto the results. While a normal artificial neural network (ANN) has input neurons which are completely connected to the the next layer's neuron (also known as an affine layer), in a CNN the layers are not connected in the same way. What happens instead is that a convolution is performed on the input layer to calculate the output. The entire input is therefore connected to the output layer's neuron. In between, every single layer affects multiple filters, and the effects of the filters are combined.

For text processing, typical inputs are characters, sentences, paragraphs or documents which are transformed into a matrix representation. Every single row of the matrix representation is representative of a token, which is typically a word but may be a character, depending on the chosen representation. Another way of understanding this is that each row in the matrix is a vector that represents a word. Normally, the aforementioned vector is a word-embedding (Word2Vec or GloVe, or a vocabulary's one-hot vector mappings). As an example, a 68-word tweet embedded in 200 dimensions would be represented as a 68 by 200 matrix as an input.

The filter used in a CNN for NLP purposes will glide over the each row of the matrix (the entire word), which results in the filter's width being similar to the input matrix's width. The region for the height of the filter is typically 2-5, representing a sliding window of 2-5 words in use. Figure 5 shows how CNN is applied on sentence classification.

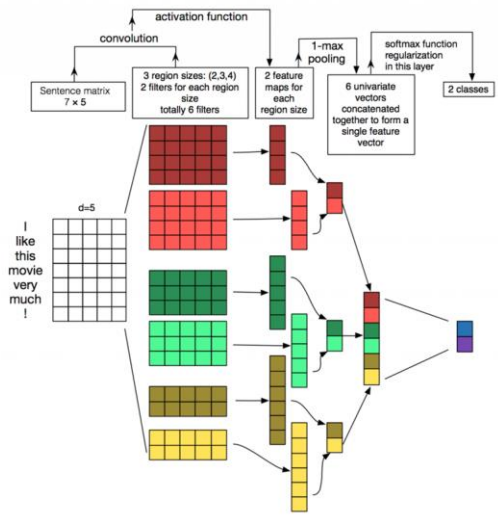

Figure 5 : CNN in use for the purpose of sentence classification [9]

A criticism of CNN is that conceptually CNN are not as intuitively applied to text as compared to CNN's application to images. Invariance and local composition are more important for images, as compared to text. In text, the positions of words relative to each other does not guarantee semantic meaning, while even in compositional sense, CNN do not appear to explain the composition of words intuitively. CNN however, does have an advantage as a representation model as it is a model that can be built quickly, as the convolution computation is hard-wired into many GPUs.

In constructing a CNN for text processing, hyper-parameters for convolution such as narrow versus wide convolution, stride size, pooling layers, and channels need to be considered for optimal results. In applying filters to the edges of a matrix, zero-padding needs to be performed. All elements outside the matrix are considered zero, and this will allow the filter to be applied on each element of the input matrix and result in an output of similar or larger size. The application of zero-padding ends up in wide-convolution, while narrow-convolution is when zero-padding is not performed. Stride size is the measure of how much is the filter shifted at each step. A larger stride size will result in a smaller output size. Typically, literature suggests that stride size should be kept at one, but there are evidence to suggest that a stride size of above 1 will lead to a model that behaves similarly to a RNN. Pooling layers are layers which subsample their input, and is commonly applied via the application of a max function to each filter's result. Pooling is essential for classification tasks, as it provides a fixed sized output. For instance, applying pooling to 1000 filters will yield a 1000-d output, regardless of filter and size of input. Pooling also achieves dimension reduction, while retaining useful information. The final aspect to be considered in the construction of CNN is the application of channels. Each channel in a CNN can used to represent different settings in terms of word embeddings, phrasing of text, or language of text.

In literature, CNN has been applied successfully onto solving problems related to classification, for example sentiment analysis, spam detection and topic modelling. [10] implemented a CNN architecture for performing sentiment analysis and topic categorization, using a concatenated word2vec word embeddings representation for text. A similar work was performed in [11]. Although it would appear that word embeddings helped, CNN has been constructed without using word embeddings, as shown in [12], which uses one-hot word vectors. Hyper-parameters and their effects on CNN was investigated thoroughly by [9].

5. Recurrent Neural Network and text

Recurrent Neural Network (RNN) can be understood as a neural network which utilizes sequential information. For a normal neural network, all inputs and outputs are assumed to be independent of each other. This approach is a bad idea once applied into the domain of text mining – any text in any language has dependency on the text that appears before a

text of interest. For instance, in the task of sentence prediction or completion, fore knowledge of existing words that appears before the target text is of paramount importance. RNNs are recurrent as a RNN parses each element of a sequence through the same set of operation, with the output dependent on the previous operation. As such, RNN can be thought of as possessing a "memory" that records information which has been computed previously. Figure 6 shows a RNN as it unfolds in time.

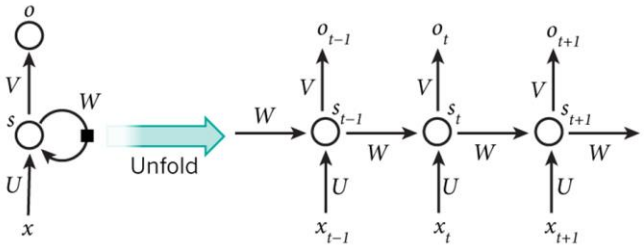

Figure 6: An RNN as it unfolds over time [1]

The input to the RNN is represented as x_t at a time step $_t$. To illustrate this further, x_1 can be a word vector representing the nth word in a sentence while s_t is the hidden state at time step $_t$. This is what we can take to mean as the 'memory' of the RNN. Finally, s_t is computed using the information in the previous hidden state and the current step's input as shown in formula 2.

$$s_t = f(Ux_t + Ws_{t-1}) \quad (2)$$

The function f in the formula above is a non-linear function such as a ReLU or a tanh function. The term s_{-1} which is necessary for the calculation of the first hidden state is normally set to zero. o_r is the output at step $_t$. For instance, if we wanted to perform a prediction of the next most probable word, o_t=softmax(Vs_t)

Parameters U, V and W are the same across every layer in an RNN, indicating that the same process is performed at every step using different inputs, thus reducing the numbers of parameters that need to be learnt by an RNN. The design of a RNN for text mining is constrained by the vocabulary size , C, and the size of the hidden layer, H. As an example, recall that the number of parameters that is needed to be learnt for a RNN is given by the following formula.

$$\text{Number of parameters} = 2HC + H^2 \quad (3)$$

Consider the case of a modest vocabulary C of size 5000, with 100 hidden layers (H =100). That would result in 1,010,000 parameters to be learnt.

Just like a normal neural network, a RNN learns by minimising the loss function applied to the training data, using for instance a cross-entropy function. A cross-entropy function is a cumulative loss function across all the examples provided in the training data. In order to minimize the total loss over the training dataset, the parameters U, V and W needs to optimized, typically by using Stochastic Gradient Descent (SGD). SGD for a RNN is normally calculated using a Backpropagation Through Time (BPTT) technique. BPTT however, are vulnerable to both the problem of vanishing and exploding gradients, which is contributed by the fact that a RNN can either remember late or early sequences well. These weaknesses have been addressed by the usage of both Long Short-Term Memory (LSTM) [13] or Gated Recurrent Unit (GRU)[14].

RNN has been used to represent language models beginning in [15], where it was proven that although RNN appears to be a simple model, it has the potential to perform competitively versus that of other models trained on far larger dataset. More recent efforts has seen a combination of CNN convoluted over text characters, from which it's output was sent to long short-term memory (LSTM) recurrent neural network language model (RNN-LM), achieving strong results although using 60% less parameters. Apart from the text domain, RNNs are being used increasingly in domains such as automated annotation of images [16], as well as in speech recognition[17].

6. Conclusion

In this paper we have performed literature review of key ideas behind the application of deep learning for the purpose of learning representation of text. A discussion on the usage of CNN and RNN is presented, with an emphasis on practical hyper-parameter options for usage of CNN is also presented.

References

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.

[3] D. Silver *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[4] A. Gray, "DIGITS: Deep Learning GPU Training System," *Parallel Forall*, 17-Mar-2015. [Online]. Available: https://devblogs.nvidia.com/parallelforall/digits-deep-learning-gpu-training-system/. [Accessed: 13-Jan-2017].

[5] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Netw.*, vol. 61, pp. 85–117, Jul. 2015.

[6] T. Luong, R. Socher, and C. D. Manning, "Better Word Representations with Recursive Neural Networks for Morphology," in *CoNLL*, 2013, pp. 104–113.

[7] R. Socher, "Deep learning for Natural Language Processing," 31-Mar-2016.

[8] S. R. Bowman, C. Potts, and C. D. Manning, "Recursive neural networks for learning logical semantics," *CoRR Abs14061827*, 2014.

[9] Y. Zhang and B. Wallace, "A Sensitivity Analysis of (and Practitioners' Guide to) Convolutional Neural Networks for Sentence Classification," *ArXiv151003820 Cs*, Oct. 2015.

[10] Y. Kim, "Convolutional Neural Networks for Sentence Classification," *EMNLP 2014*, 2014.

[11] N. Kalchbrenner, E. Grefenstette, and P. Blunsom, "A convolutional neural network for modelling sentences," *ArXiv Prepr. ArXiv14042188*, 2014.

[12] R. Johnson and T. Zhang, "Effective use of word order for text categorization with convolutional neural networks," *ArXiv Prepr. ArXiv14121058*, 2014.

[13] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to Sequence Learning with Neural Networks,” *ArXiv14093215 Cs*, Sep. 2014.
[14] M. Sundermeyer, R. Schlüter, and H. Ney, “LSTM Neural Networks for Language Modeling.,” in *Interspeech*, 2012, pp. 194–197.
[15] T. Mikolov, M. Karafiát, L. Burget, J. Cernockỳ, and S. Khudanpur, “Recurrent neural network based language model.,” in *Interspeech*, 2010, vol. 2, p. 3.
[16] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, “Show and Tell: A Neural Image Caption Generator,” presented at the Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 3156–3164.
[17] H. Sak, A. Senior, K. Rao, and F. Beaufays, “Fast and Accurate Recurrent Neural Network Acoustic Models for Speech Recognition,” *ArXiv150706947 Cs Stat*, Jul. 2015.

View publication stats